

Retrieval-Augmented Generation

Embeddings • Vector Databases • Knowledge Retrieval

From agentic action to reliable access to organizational knowledge

What You Will Learn Today



1. External Knowledge

Explain why an LLM requires external, private organizational documents beyond its pre-trained cutoff.



2. Vector Embeddings

Understand how text is converted into dense numerical vectors to measure semantic similarity.



3. Chunking & Metadata

Master document splitting strategies, chunk overlap, and attaching rich metadata tags.



4. Vector Databases

Learn how specialized vector databases store embeddings and execute high-speed similarity searches.



5. Retrieval Pipeline

Understand query vectorization, cosine similarity metrics, and tuning top-k document retrieval.



6. Agentic Integration

Connect RAG retrievers to our Week 3 LangGraph agent as an authoritative tool.

Recall: How Our Application Evolved

WEEK 1-2

LLM Baseline

Single Direct Prompt

- Generate an answer purely from pre-training weights.
- No external data, state, or tools.
- Risk of severe hallucinations on private topics.

WEEK 3

Agent + Tools + Memory

LangGraph Orchestration

- Plan, remember user intent, and execute actions.
- Query databases and trigger APIs.
- Solves the **ACTION** problem in enterprise workflows.

WEEK 4 (TODAY)

+ RAG System

Grounding Knowledge

- Retrieve verified company documents before answering.
- Dynamic knowledge access at query time.
- Solves the **KNOWLEDGE ACCESS** problem.

Motivation: The Knowledge Access Problem

Daily Essentials Scenario

Our customer support system has hundreds of private internal documents:

- Return Policies
- Refund Rules
- Warranty FAQs
- Shipping Specs
- Product Manuals

CUSTOMER QUERY:

"My mixer stopped working after 20 days. Can I return it or get a replacement?"

Why LLMs Fail Here Alone

- The LLM does **not automatically know** our private company documents.
- Fine-tuning on changing policy docs is slow, expensive, and prone to hallucinated details.
- We need an automated way to **retrieve relevant knowledge at query time**.

**THE GROUNDING
PATTERN**

Customer Question → Retrieve Policy → Supply Context → LLM Answer

Tools vs. Retrieval-Augmented Generation



Structured Tools

Solves the ACTION Problem

- Executes stateful operations (place order, refund payment).
- Queries structured relational databases (SQL, APIs).
- Deterministic input/output formats.

Question → Tool → Database → Action



RAG Retrieval

Solves KNOWLEDGE ACCESS

- Finds relevant text snippets inside unstructured documents.
- Searches policy manuals, product guides, FAQs, and PDFs.
- Passes retrieved text as dynamic context to the LLM.

Question → Retriever → Documents → Answer



Key Insight: Modern production AI agents combine BOTH — Tools for state changes, RAG for authoritative knowledge!

What is Retrieval-Augmented Generation?

1. Retrieval

Search external knowledge bases to find relevant document chunks matching the user's specific query.

2. Augmentation

Inject retrieved document context directly into the prompt payload passed to the Large Language Model.

3. Generation

The LLM reads the user prompt + retrieved context to produce an accurate, factually grounded answer.

The Core RAG Promise

Instead of expecting the LLM to memorize all world & company data in pre-training weights, we supply exact source facts at inference time.

SYSTEM BENEFIT

Zero hallucination, fully auditable citations, instant document updates!

LLM-Only vs. RAG Pipeline

Baseline: LLM-Only Pipeline

Unanchored

User Question



Base LLM (Parametric Memory)



Generated Answer (High Risk of Hallucination)

RAG Architecture Pipeline

Context Grounded

User Question



Retriever (Vector DB Search)



Relevant Knowledge Chunks



LLM Generator



Factual Grounded Answer

Concrete Example: Return Policy Query

Step 1: User Question

"What is the return policy for electronics?"

Step 2: Vector DB Retrieval

Retrieved Context (Chunk #104):

"Electronics purchased from Daily Essentials may be returned within 15 days of delivery provided they are unopened in original packaging..."

Step 3: Prompt Augmentation

Payload Sent to LLM:

System: Answer using ONLY the context provided. Context: [Chunk #104 content] User: What is the return policy for electronics?

Step 4: Grounded Response

✓ "According to Daily Essentials policy, electronics can be returned within 15 days of delivery if unopened in original packaging."

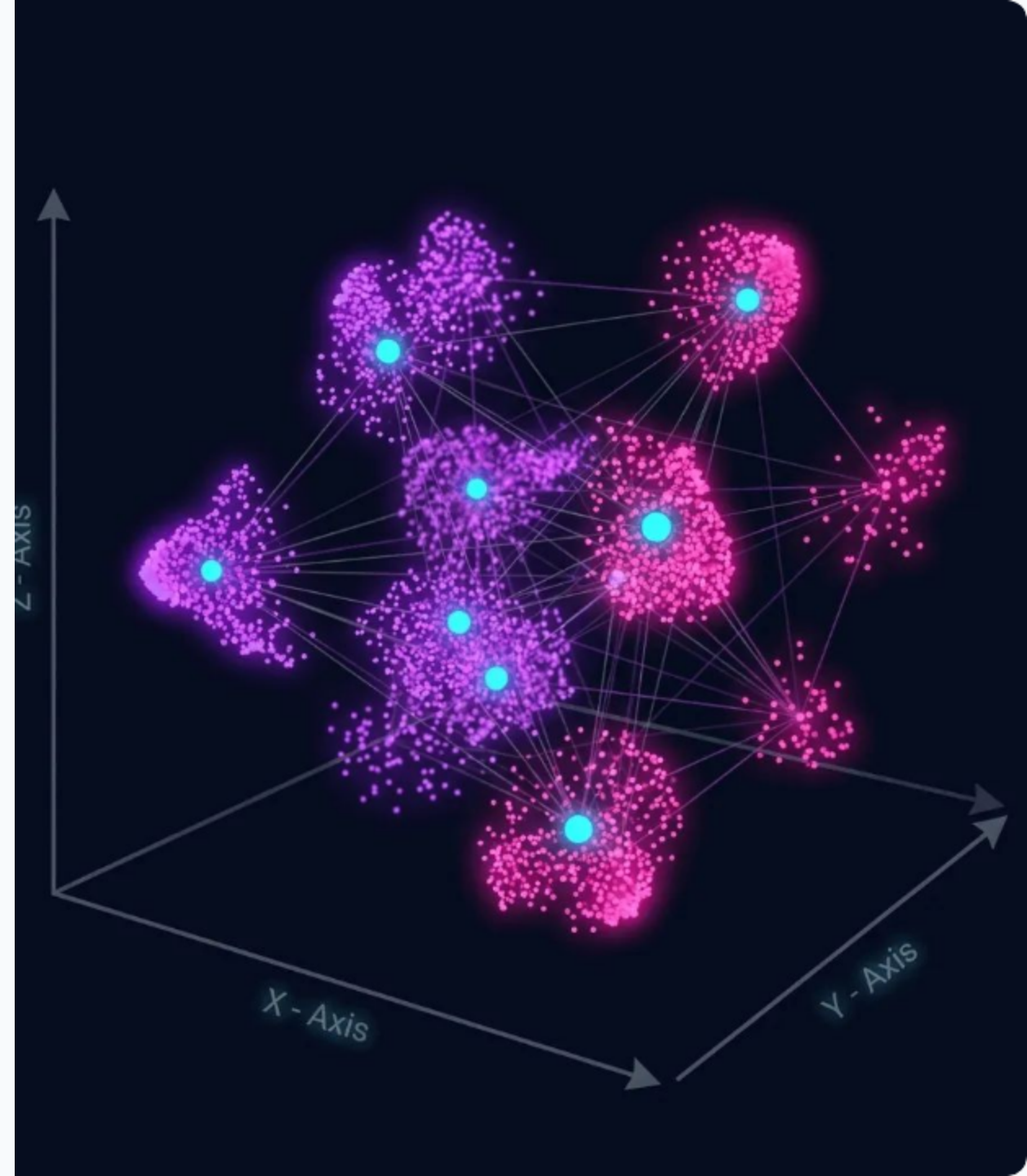
Embeddings: Text as Vectors

An **embedding** is a high-dimensional numerical vector that represents the semantic meaning of text.

TRANSFORMATION PIPELINE

Text → Embedding Model → Vector [0.024, -0.198, ..., 0.852]

- Semantically similar phrases map to points close to each other in vector space.
- Captured geometry allows mathematical comparison of human concepts.



Semantic Similarity in Action

Matching Meaning Over Keywords

Keyword search fails when users use different terminology. Vector embeddings solve this naturally:

"How can I get my money back?"
"I want a refund."

0.92 (High)

"How can I get my money back?"
"Mixer motor specs 120V."

0.14 (Low)



High-Dimensional Clusters

Documents with similar themes (e.g., *reimbursements*, *damaged goods*, *shipping delays*) form distinct clusters in 1536-dimensional space.

How Computers Compare Meaning

1

Query Vector

Convert user question into vector $\vec{q}$

2

Index Compare

Compare against chunk vectors $\vec{d}$

3

Cosine Metric

Compute angle distance between vectors

4

Top Score

Return closest matches to LLM

COSINE SIMILARITY FORMULA

$$\text{Similarity} = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

A cosine score of **1.0** represents identical semantic trajectory; **0.0** represents orthogonality (no relevance).

What is a Vector Database?

Core Capabilities

- Stores millions of high-dimensional vector embeddings efficiently.
- Executes sub-millisecond Approximate Nearest Neighbor (ANN) search.
- Stores corresponding text chunks and key-value metadata tags.
- Supports dynamic payload filtering (e.g., filter by department before search).

Popular Vector Databases

Qdrant

Rust-based, high performance, rich filtering.

Pinecone

Fully managed cloud-native vector index.

Chroma

Lightweight open-source embeddable database.

pgvector

Vector extension for PostgreSQL instances.

Relational DB vs. Vector Search

Feature	Relational DB (SQL)	Vector Search (ANN)
Search Paradigm	Exact keyword / primary key match	Semantic proximity / distance metric
Query Type	SELECT * WHERE id = 104	find_nearest("broken blender return")
Handling Typos/Synonyms	Fails unless explicitly indexed	Handles synonyms & rephrasing naturally
Data Type	Structured tables, numbers, strings	Unstructured text, PDFs, audio, vectors
Role in Architecture	Order states, user accounts, transactional tool	Retrieval-augmented knowledge grounding

Document Chunking Strategies

Why Split Documents?

- LLMs have context window limits & attention degradation on giant texts.
- Embedding a 100-page PDF into 1 vector loses specific paragraph details.
- Smaller chunks provide surgical precision during top-k retrieval.

Common Chunking Methods

Fixed-Size Chunking: Split every N characters/tokens (e.g., 500 tokens).

Recursive Splitting: Split by paragraphs, then sentences, then words.

Document-Aware: Respect Markdown headers, HTML tags, or PDF sections.

Chunk Overlap & Rich Metadata

Chunk Overlap

Prevents crucial sentences from being sliced in half at a chunk boundary:

[Chunk 1]: "Daily Essentials allows returns within 30 days. Defective appliances qualify for free replacement..."

[Overlap Region]: "...Defective appliances qualify for free replacement..."

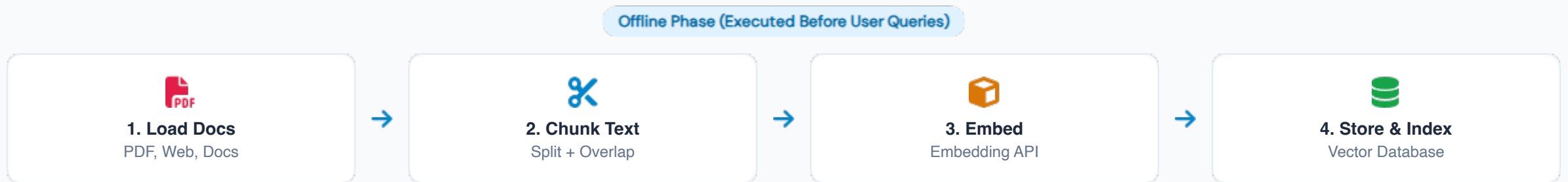
[Chunk 2]: "...Defective appliances qualify for free replacement. Return shipping labels are pre-paid."

Metadata Payload

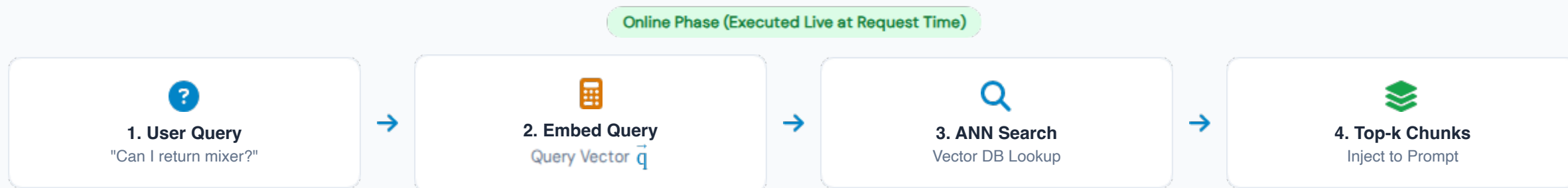
Attaching attributes enables filtered vector retrieval:

```
{ "chunk_id": "doc_882_c4", "source": "warranty_policy_2026.pdf",  
  "category": "Returns", "department": "Electronics", "page_number": 3,  
  "last_updated": "2026-01-15" }
```

The Ingestion & Indexing Pipeline



The Query Retrieval Pipeline



Understanding Top-k Search & Tuning

What is k ?

k represents the exact number of top-scoring relevant chunks extracted from the vector index and supplied to the LLM.

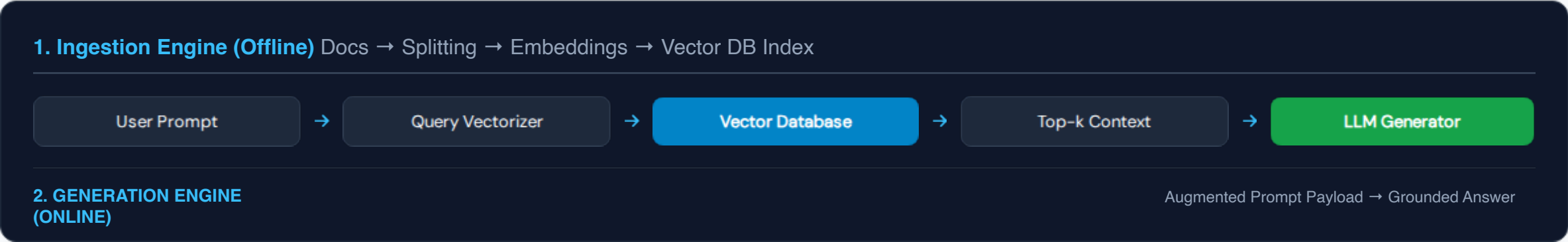
Common Setting: $k = 3$ to 5

Tuning Trade-offs

Too Small ($k=1$): Fast & cheap, but risks missing secondary context.

Too Large ($k=20$): High token costs, risk of prompt distraction ("lost in the middle").

End-to-End RAG Architecture



Connecting RAG to LangGraph Agent

RAG as an Agent Tool

RAG does **not** replace the agentic loop. Instead, the Vector Retriever is wrapped as a callable tool inside LangGraph!

```
@tool def retrieve_policy_docs(query: str) → str: """Searches Daily  
Essentials company policies.""" docs =  
vector_db.similarity_search(query, k=3) return  
"\n".join([d.page_content for d in docs])
```

LangGraph Workflow State

- **Agent Decision Node:** Reads user query and determines whether to call SQL Order tool or RAG Policy tool.
- **Tool Execution Node:** Executes Vector DB similarity search and appends chunks to State.
- **LLM Response Node:** Synthesizes order data + policy chunks into final reply.

Execution Trace: Agent + Tools + RAG

User Query: "My mixer order A1031 arrived 20 days ago and stopped working. Can I get a replacement?"

[Step 1 – Order Tool]: Call `fetch_order_details("A1031")`
Result: Product = Stand Mixer, Delivered = 20 Days Ago.

[Step 2 – RAG Policy Tool]: Call `retrieve_policy("defective appliance exchange policy")`
Result: Chunk #88: "Defective electronics qualify for replacement within 30 days."

[Step 3 – Final Synthesis]:
"Yes! Since order A1031 was delivered 20 days ago, it falls within our 30-day defective exchange window. I can initiate your replacement now."

What RAG Solves — and What It Doesn't

✓ What RAG Solves

- Eliminates model retraining for document updates.
- Provides exact enterprise context at query time.
- Enables auditable source document citations.
- Prevents hallucinations on factual domain queries.

✗ What RAG Doesn't Fix

- Garbage In, Garbage Out (bad documents = bad answers).
- Poor chunking or missing metadata tags.
- Irrelevant top-k retrieval scores.
- Complex multi-step reasoning failures.

Course Story: Week 1 to Week 5

WEEK 1

LLM Basics

Prompting, temperature, baseline generation.

WEEK 3

Agents & Tools

LangGraph, state management, SQL tool calling.

WEEK 4 (TODAY)

RAG Systems

Embeddings, vector DBs, knowledge retrieval.

WEEK 5 (NEXT)

LLM/Transformers

Attention mechanism, model architecture internals.

Lecture 4 Key Takeaways

- **RAG Pattern:** Retrieve context → Augment prompt → Grounded generation.
- **Vector Mechanics:** High-dimensional embeddings capture semantic meaning beyond exact keywords.

- **Vector Databases:** Perform sub-millisecond similarity search over chunked text + metadata.
- **Agent Synergy:** RAG serves as an authoritative knowledge tool inside agentic workflows.

Next Week: Opening the Transformer Black Box



Week 5 Preview: Transformer Internals

We have built agents and RAG pipelines using LLMs as black boxes. Next week, we look under the hood to see how Attention Mechanisms turn tokens into embeddings!